

原始请求 · curl	Raw HTTP	消息 & 参数	Schema	Python SDK · 同步	openai	Python SDK · 异步	AsyncOpenAI																													
基础 chat 请求 <pre>curl https://api.openai.com/v1/chat/completions \ -H "Authorization: Bearer \$OPENAI_API_KEY" \ -H "Content-Type: application/json" \ -d '{ "model": "gpt-5", "messages": [{"role": "system", "content": "You are concise."}, {"role": "user", "content": "Hello"}], "temperature": 0.7, "max_tokens": 1024 }'</pre> 流式 SSE <pre>curl .../v1/chat/completions \ -H "Authorization: Bearer \$KEY" \ -d '{"model": "gpt-5", "messages": [...], "stream": true, "stream_options": {"include_usage": true}}'</pre> # 响应: 每行 data: {...}\n\n, 以 data: [DONE] 结束		消息角色 <table><tr><td>system</td><td>身份/约束(单条,放最前)</td></tr><tr><td>developer</td><td>Responses API 替代 system</td></tr><tr><td>user</td><td>用户输入</td></tr><tr><td>assistant</td><td>模型回复 / 历史</td></tr><tr><td>tool</td><td>工具结果(配 tool_call_id)</td></tr></table> 采样 <table><tr><td>temperature</td><td>0-2,默认 1,确定性用 0</td></tr><tr><td>top_p</td><td>核采样,与 temp 二选一</td></tr><tr><td>max_tokens</td><td>输出上限</td></tr><tr><td>seed</td><td>尽量确定性(非保证)</td></tr><tr><td>frequency_penalty</td><td>降重复</td></tr><tr><td>presence_penalty</td><td>鼓励新话题</td></tr><tr><td>stop</td><td>停止序列(最多 4)</td></tr><tr><td>n</td><td>生成几个候选</td></tr><tr><td>logprobs / top_logprobs</td><td>概率信息</td></tr><tr><td>user</td><td>追踪用户 ID(滥用检测)</td></tr></table>		system	身份/约束(单条,放最前)	developer	Responses API 替代 system	user	用户输入	assistant	模型回复 / 历史	tool	工具结果(配 tool_call_id)	temperature	0-2,默认 1,确定性用 0	top_p	核采样,与 temp 二选一	max_tokens	输出上限	seed	尽量确定性(非保证)	frequency_penalty	降重复	presence_penalty	鼓励新话题	stop	停止序列(最多 4)	n	生成几个候选	logprobs / top_logprobs	概率信息	user	追踪用户 ID(滥用检测)	安装 & 客户端 <pre>pip install openai</pre> 基本调用 <pre>from openai import OpenAI client = OpenAI(api_key="sk-...", base_url="https://api.openai.com/v1", # 可换兼容端 timeout=60.0, max_retries=2,) resp = client.chat.completions.create(model="gpt-5", messages=[{"role": "user", "content": "Hi"}], temperature=0.3,) print(resp.choices[0].message.content) print(resp.usage.total_tokens)</pre> 流式 <pre>stream = client.chat.completions.create(model="gpt-5", messages=[...], stream=True, stream_options={"include_usage": True},) for chunk in stream: delta = chunk.choices[0].delta.content if delta: print(delta, end="", flush=True)</pre>		异步客户端 <pre>import asyncio from openai import AsyncOpenAI client = AsyncOpenAI(api_key="sk-...") async def ask(q): r = await client.chat.completions.create(model="gpt-5", messages=[{"role": "user", "content": q}],) return r.choices[0].message.content asyncio.run(ask("hi"))</pre> 并发批量 <pre>async def batch(qs): tasks = [ask(q) for q in qs] return await asyncio.gather(*tasks) # 或限流: asyncio.Semaphore(10)</pre> 异步流 <pre>stream = await client.chat.completions.create(model="gpt-5", messages=[...], stream=True,) async for chunk in stream: d = chunk.choices[0].delta.content if d: print(d, end="", flush=True)</pre> 资源回收 <pre>await client.close() 关闭 httpx 连接池 async with AsyncOpenAI(...) as client: 推荐 context manager</pre>
system	身份/约束(单条,放最前)																																			
developer	Responses API 替代 system																																			
user	用户输入																																			
assistant	模型回复 / 历史																																			
tool	工具结果(配 tool_call_id)																																			
temperature	0-2,默认 1,确定性用 0																																			
top_p	核采样,与 temp 二选一																																			
max_tokens	输出上限																																			
seed	尽量确定性(非保证)																																			
frequency_penalty	降重复																																			
presence_penalty	鼓励新话题																																			
stop	停止序列(最多 4)																																			
n	生成几个候选																																			
logprobs / top_logprobs	概率信息																																			
user	追踪用户 ID(滥用检测)																																			

Tool Calling

函数调用

定义工具

```
tools = [{
  "type": "function",
  "function": {
    "name": "get_weather",
    "description": "Get city weather",
    "parameters": {
      "type": "object",
      "properties": {
        "city": {"type": "string"}
      },
      "required": ["city"],
      "additionalProperties": false
    },
    "strict": true
  }
}]
```

控制

```
tool_choice="auto" # 默认
tool_choice="required" # 必须调工具
tool_choice={"type": "function", "function": {"name": "X"}} # 指定
parallel_tool_calls=True # 默认并行
```

回灌结果

```
messages.append(resp.choices[0].message) # tool_calls
for call in resp.choices[0].message.tool_calls:
    result = run_tool(call)
    messages.append({
      "role": "tool",
      "tool_call_id": call.id,
      "content": str(result),
    })
# 再 create 一次拿最终回复
```

结构化 & 多模态

JSON Schema(Structured Outputs)

```
from pydantic import BaseModel
class User(BaseModel):
    name: str
    age: int

resp = client.chat.completions.parse(
    model="gpt-5",
    messages=[...],
    response_format=User,
)
user = resp.choices[0].message.parsed
```

JSON mode(老式)

```
response_format={"type": "json_object"}
```

Vision · 图片

```
messages = [
  {
    "role": "user",
    "content": [
      {"type": "text", "text": "What is this?"},
      {"type": "image_url",
        "image_url": {"url": "data:image/png;base64,..."}}
    ]
  }
]
```

音频 / 文件

input_audio	gpt-4o–audio
file_id	Files API 上传

错误 & 兼容端

Errors & Providers

HTTP 状态码

401	AuthenticationError · key 错
403	PermissionDeniedError · 区域/scope
404	model 不存在
429	RateLimitError · 看 Retry–After
400	BadRequestError · schema 错
500/503	服务端,可重试

重试与超时

```
OpenAI(max_retries=3) SDK 自动指数退避
timeout=httpx.Timeout(10) 粒度超时
tenacity 外部重试库
```

兼容端 base_url 速记

Ollama	http://localhost:11434/v1
vLLM	http://host:8000/v1
DeepSeek	api.deepseek.com
Kimi(月之暗面)	api.moonshot.cn/v1
Qwen / DashScope	...aliyuncs.com/compatible-mode/v1
Groq · xAI · Together	官方文档 base_url
OpenRouter	openrouter.ai/api/v1 多模型聚合

Pro Tips

实战

- 异步优先: 并发 10× 用 AsyncOpenAI + asyncio.gather + Semaphore
- SSE 计数: 加 stream_options.include_usage=true, 最后 chunk 带 usage
- Structured Outputs 用 .parse() + Pydantic,**别**用旧 JSON mode 拼 prompt
- Tools 加 strict: true 严格 schema,模型不再瞎编字段
- Prompt cache 自动: >1024 token 前缀复用,无需配置;dashboard 看命中率
- Batch API 50% 折扣: 非实时任务走 /v1/batches, 24h 完成
- 速率限 429: 看 x-ratelimit-* header 退避
- Responses API(/v1/responses)是新一代,内置 stateful + tools,新项目推荐
- 多家切换用 LiteLLM / portkey 统一接口、统一计费
- 本地兼容端少有 logprobs / seed 严格语义,提前确认
- 不要把 key 硬编码,用 os.environ + .env
- Function call 的 arguments 是 JSON 字符串,记得 json.loads()